

Nile University (NU) IoT Research Internship Report

Day 2

Overview

Day 2 moved from simple digital input/output control into more advanced GPIO handling, sensor integration, and data management. The day progressed in a clear line: replacing inefficient input detection with interrupts, adding a real sensor to generate meaningful data, storing that data persistently, and finally visualizing it.

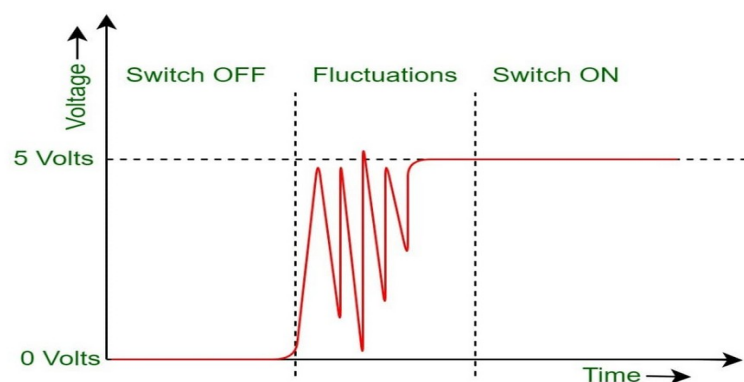
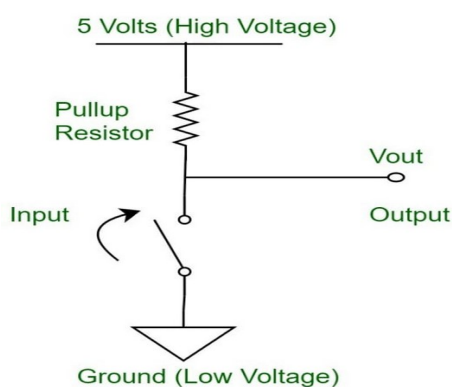
Task 1: Interrupt-Driven Button with Debouncing

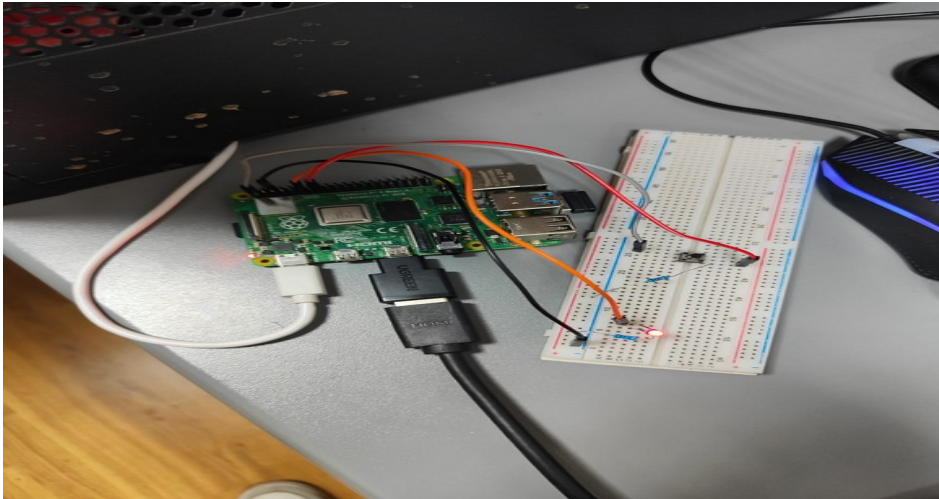
Replaced the polling-based button detection from Day 1 with hardware interrupts. Instead of continuously checking the button state in a loop, the GPIO pin was configured to trigger an interrupt on a falling edge (the moment the button is pressed).

Key elements:

- Falling edge detection configured using `gpio_line_request_falling_edge_events()`
- The program blocks on `gpio_line_event_wait()`, consuming 0% CPU until a press occurs
- Software debouncing implemented with a 200ms window using timestamp comparison, filtering out the multiple rapid transitions a mechanical button produces when pressed
- LED toggle logic: each valid press flips the LED state (ON/OFF) using `gpio_line_set_value()`
- Hardware setup: button on GPIO 17 (pull-up resistor), LED on GPIO 27, both wired through appropriate resistors

Improvement over Day 1: on Day 1, the button was read using polling, which constantly uses CPU cycles to check the pin state. This task solved that inefficiency directly by moving to an interrupt-driven, event-based approach, which was the exact next step Day 1 identified as an improvement to pursue.





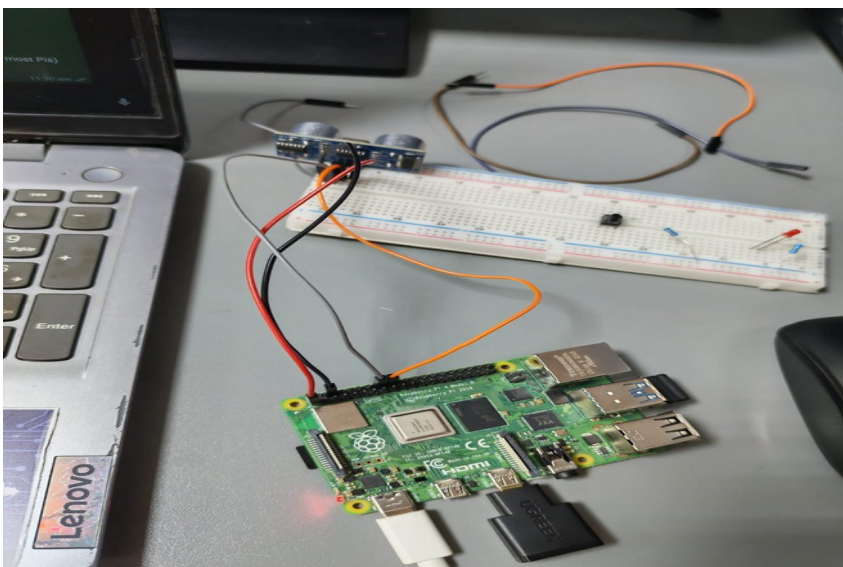
Task 2: Ultrasonic Sensor (HC-SR04)

Integrated an HC-SR04 ultrasonic distance sensor to measure the distance to nearby objects. The sensor emits ultrasonic waves and measures the time taken for the echo to return.

Key elements:

- Trigger pin (GPIO 23) sends a $10\mu\text{s}$ HIGH pulse to start a measurement
- Echo pin (GPIO 24) receives a pulse whose duration equals the round-trip travel time of the sound wave
- Distance calculated as $(\text{Speed of Sound} \times \text{Time}) / 2$
- Used the `gpiozero` library's `DistanceSensor` class to simplify timing and pulse measurement
- Distance converted from meters to centimeters for readability, with readings taken every 2 seconds

Improvement over Task 1: Task 1 dealt with simple binary input (button pressed or not). This task introduced analog-style measurement from a real-world sensor, moving the work from digital signal handling into actual sensor data acquisition, which is a core IoT skill.



```

Intern@Intern:~/tasks $ python3 task5-2.py
=====
LAST 10 SENSOR READINGS
=====
ID      | Timestamp                | Distance
-----|-----|-----
19      | 2026-07-21 12:28:31     | 10.73 cm
18      | 2026-07-21 12:28:29     | 10.68 cm
17      | 2026-07-21 12:28:27     | 10.69 cm
16      | 2026-07-21 12:28:25     | 10.72 cm
15      | 2026-07-21 12:28:23     | 10.70 cm
14      | 2026-07-21 12:28:21     | 10.71 cm
13      | 2026-07-21 12:28:19     | 10.67 cm
12      | 2026-07-21 12:28:17     | 10.66 cm
11      | 2026-07-21 12:28:15     | 10.69 cm
10      | 2026-07-21 12:28:13     | 10.73 cm
=====
STATISTICS ON THESE 10 READINGS
=====
-> Sample Size:      10 readings
-> Average Distance: 10.70 cm
-> Closest Object:   10.66 cm
-> Furthest Object:  10.73 cm

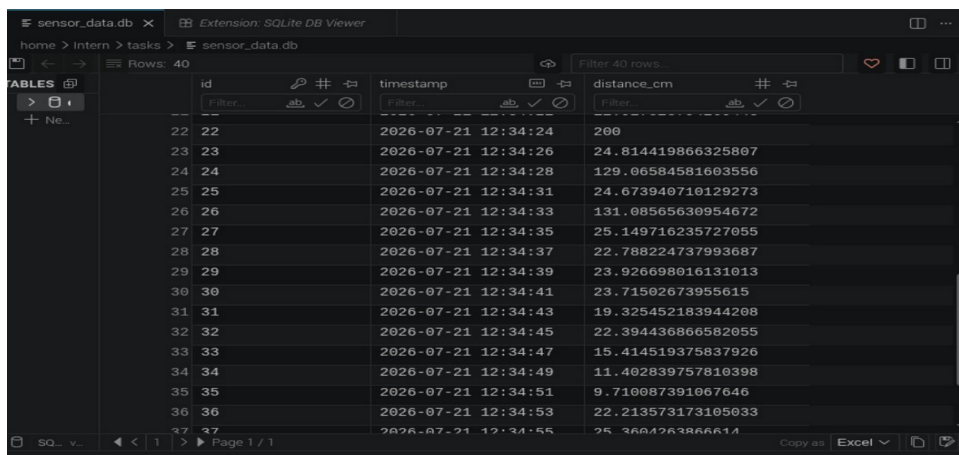
```

Task 3: SQLite Database Integration

Built a persistent storage system to log every sensor reading for later analysis, using a `distance_logs` table with an auto-incrementing ID, a timestamp, and the distance in centimeters. The table is created automatically on first run.

SQLite was chosen because it is lightweight, serverless, requires no external dependencies, and is well suited to embedded systems while still being easy to query and export.

Improvement over Task 2: Task 2 only produced live readings that were visible momentarily and then lost. This task solved that by giving the readings a permanent record, which is necessary before any meaningful analysis or trend-tracking can happen.



```

Intern@Intern:~/tasks $ python3 task5.py
/usr/lib/python3/dist-packages/gpiozero/input_devices.py:852: PWMSoftware
pin factory. See https://gpiozero.readthedocs.io/en/stable/api_input.html#
warnings.warn(PWMSoftwareFallback(
Initializing Database...
Starting sensor monitoring. Press Ctrl+C to stop.
logged data200.0
[2026-07-21 12:34:20] Logged: 200.00 cm
logged data12.927823794269443
[2026-07-21 12:34:22] Logged: 12.93 cm
logged data200.0
[2026-07-21 12:34:24] Logged: 200.00 cm
logged data24.814419866325807
[2026-07-21 12:34:26] Logged: 24.81 cm
logged data129.06584581603556
[2026-07-21 12:34:28] Logged: 129.07 cm
logged data24.673940710129273
[2026-07-21 12:34:31] Logged: 24.67 cm
logged data131.08565630954672
[2026-07-21 12:34:33] Logged: 131.09 cm

```

Tools & Libraries Used

C Programming:

- `libgpiod` for GPIO control
- `time.h` for timestamp calculations
- `unistd.h` for standard POSIX functions

Python Programming:

- `gpiozero` for GPIO control
- `sqlite3` for database management
- `datetime` for timestamp generation
- `matplotlib` for data visualization
- `pandas` for data manipulation
- `time` for timing control

Outcome

- Replaced the inefficient polling method from Day 1 with a proper interrupt-driven, debounced button input
- Successfully integrated a real distance sensor (HC-SR04) and understood the physics behind its timing-based measurement
- Built a working SQLite logging system to persist sensor data
- Completed a full end-to-end IoT data pipeline on this day: sense → store → visualize